

Self-Drafting Qwen3.8-27B at Two Bits: Speed, Energy, and Memory on a 12 GB Laptop GPU

Matthew Schwartz

Independent researcher [ORCID 0009-0009-4171-7247](https://orcid.org/0009-0009-4171-7247) matthew.schwartz.research@gmail.com

September 2026

Abstract

Self-drafting can accelerate a memory-constrained language model while consuming memory needed for context. We measure that tradeoff for Qwen3.8-27B on a 12 GB RTX 4080 Laptop GPU capped at 80 W, using a 2.00-bit EXL3 quantization and low-bit GGUF comparators. With its own multi-token-prediction head, EXL3 completion throughput rises from about 29 to 48.7–50.4 tokens/s on code and math and 35.7 on prose. Sustained drafted runs deliver 0.559 tokens/J for EXL3 and 0.577 for GGUF, measured from GPU-board power. Reducing the server’s concurrency allocation from four slots to one lowers drafted load memory from 10,226 to 8,050 MiB. Drafted accuracy is 97.6 versus 97.2% on 500 paired GSM8K items and 86.8 versus 86.2% on 500 paired MMLU items at temperature zero; these observations show no measured decrease but do not establish equivalence. Source-level diagnostics explain the recurrent-state allocation and identify prefill and draft acceptance bottlenecks in the tested engines. Exposing an existing checkpoint interval also reduces one edited-prompt latency from 10.1 to 5.0s, at a cost in host memory and cold prefill. This is a deployment benchmark with configuration changes, not a new decoding method. Results describe one loaded laptop and pinned software; most EXL3 measurements used a mixed CUDA library environment, and performance on newer releases is untested.

1 Introduction

On a 12 GB GPU, the memory used to accelerate a quantized model competes with its context window. Qwen3.8-27B makes this tradeoff concrete: its native multi-token-prediction (MTP) head proposes tokens for the main model to verify, but speculative rollback also requires stored recurrent states. The useful deployment question is whether the speed and energy gains justify that memory, and which serving settings change the tradeoff.

Prior work already establishes speculative decoding and its energy benefits [1–3]. Author benchmarks also measure Qwen3.8-27B with MTP on desktop RTX 3090 cards, including EXL3 and GGUF deployments [4–6]. Their different precision, power, prompts and software do not determine the result on this 80 W laptop. Our question is narrower: what speed, energy, memory and tested accuracy does self-drafting deliver for the low-bit artifacts that fit this machine?

For the 2.00-bpw EXL3 artifact, MTP increases completion throughput from about 29 to 48.7–50.4 tokens/s on code and math and 35.7 on prose. Sustained EXL3 and GGUF runs measure 0.559 and 0.577 tokens/J; without replicated sustained runs, that small difference is not an established ranking. Allocating one concurrent job rather than the server’s four-slot default saves 2,176 MiB on the drafted EXL3 load. Paired tests on 1,000 greedy GSM8K and MMLU items show no measured accuracy decrease, while exposing a separate run-to-run reproducibility problem in the tested exllamav3 1.4.7 build.

The contribution is a hardware-specific benchmark and a set of tested configuration changes, backed by archived measurements and source-level diagnostics. A checkpoint-interval experiment led to a merged TabbyAPI configuration interface; a nondeterminism report led to a maintainer fix checked here on 100 items. We do not propose MTP, recurrent checkpointing, or a new quantizer, and do not claim that the tested settings remain optimal as engines change. Results are organized by deployment question; the appendix retains the experiment identifiers, detailed tables and failed predictions needed to audit the evidence.

2 Experimental Setup

Hardware and operating conditions. One laptop: an NVIDIA RTX 4080 Laptop GPU with 12,282 MiB of device memory (E12) under an 80 W power limit, an Intel Core i9-14900HX, and 62 GiB of RAM. The desktop session holds about 485 MiB of VRAM (E23), leaving about 11.5 GiB, of which the campaign kept roughly 10.5 GiB as its working envelope for a model process (E12). The machine was not idle: foreign agents ran throughout and one foreign Python job held about four CPU cores, so every row is “loaded machine” unless stated. GPU power, clocks, and temperature were sampled with `nvidia-smi` at 1–2 s; energy is GPU-board energy only, with no wall meter, and idle draw was measured for 30 s with the model loaded before each sustained run (E25).

Model. Qwen3.8-27B is a dense hybrid of the Qwen3.5 family [7]: 64 layers of which 48 are Gated-DeltaNet (GDN) linear-attention layers [8] and 16 full attention, a 248,320-token vocabulary, and a one-layer MTP head that can draft for its own decoder. Each GDN layer holds a recurrent state of 48 value heads $\times$ 128 $\times$ 128 in fp32, 3 MiB per layer and about 148 MiB per sequence (E28); the attention layers’ Q8 KV cache costs 8.5 MiB per 256-token page. Both engines keep the recurrent state in fp32 (source check, H16; cf. 9). The model runs in thinking mode with the vendor’s recommended sampling (temperature 1.0, top- p 0.95, top- k 20) and `reasoning_effort: medium` unless stated.

Quantized artifacts. Table 1 lists the files. All eleven local Qwen3.8-27B artifacts were hashed and matched to the Hub’s LFS SHA-256 records on 2026-09-06 (`hash_audit_2026-09-06.json`); the Unsloth repository was re-cut on 2026-08-20, so the hashes rather than the names identify the evidence. The EXL3 file is turboderp’s `SC_2.00bpw_H3` branch [10]: a trellis-coded quantization in the QTIP family [11] with 14 tensors at 1 bit, 334 at 2, 38 at 3, and 14 at 4, a 3-bit output head, and a 2-bit MTP layer, calibrated on 217 reasoning traces (613K tokens) generated by the model’s own 6-bit quantization (E32). The GGUF files are Unsloth’s dynamic quantizations [12]; their MTP head ships as a separate 1.28 GiB sidecar, about 80% of it duplicated target tensors that a lab patch lets a 0.25 GiB stripped sidecar borrow instead (E12). The DFlash2 block drafter on the UD-IQ2_XXS lane is z-lab’s Q4_K_M GGUF of the DFlash architecture [13, 14].

Engines and patches. The EXL3 lane is TabbyAPI [15] on exllamav3 1.4.7 [16], staged in isolation from the lab’s pinned 1.4.3 (undrafted decode identical within -0.2 to $+1.5\%$ on four prompts, E7; 1.4.7 fixes a slot leak in the draft path), with a Q8 cache, `chunk_size` 1,024 (2,048 in the early 16K runs), and `draft_mode: mtp` with dynamic draft sizing. The GGUF lane is llama.cpp v0.4.0 [17] (commit 5266f24, CUDA 12.6, sm_89) with the pinned 2026-08-26 build (035e227) as a control; the builds agree within 1% on undrafted decode and short prefill (E1/E2, three llama-bench repeats). GGUF runs used `-ngl 99 -np 1 -fa on -ctk q8_0 -ctv q8_0 -ctxcp 4 -cms 1024 -b 512 -ub 256` (E4a). Four lab patches, off unless their environment variable is set, instrument or extend the staged engines (Section 8).¹ The single-user EXL3 configuration the

¹A fifth, a CPU fallback for a CUDA Cholesky failure met while converting the E32, E41, and E42 quant on this box, was withdrawn on 2026-09-10 as a non-bug once the failure was traced to the library-path fault described next

Artifact	Source	Bytes	Precision*	Resident (MiB)	Condition
EXL3 SC_2.00bpw_H3	turboderp	10,196,126,231	2.00 (+3-bit head)	7,794 / 8,050	12K undrafted (E27) / 12K drafted, 1 slot (E28)
EXL3 SC_2.20bpw_H3_V3	turboderp	10,280,960,404	2.20	8,242	8K undrafted (E23c)
GGUF UD-IQ2_S	unsloth	8,371,970,048	2.49	8,356	8K undrafted (E3)
GGUF UD-IQ2_XXS	unsloth	7,266,070,528	2.16	7,248	8K undrafted (E3)
GGUF UD-Q4_K_XL	unsloth	17,559,178,144	5.22	—	reference only; 30 layers on GPU (E40)
MTP sidecar Q4_0	unsloth	1,369,590,656	—	1,124	cost beside IQ2_S; stripped variant 0.25 GiB (E3, E12)
DFlash2 drafter Q4_K_M	z-lab	1,143,006,816	—	2,462	cost beside IQ2_XXS at 8K (E3)

Table 1: Artifacts. Bytes and SHA-256 matches are from the 2026-09-06 hash audit. *GGUF values retain the campaign’s whole-file bits divided by an approximate 26.9-billion base-model parameter count; this is not the stored-tensor denominator used in some other reports. For example, the same Q4 file gives 5.14 rather than 5.22 bits when divided by 27.3207 billion stored parameters. EXL3 values are the branch’s nominal precision of its quantized tensors, not whole-file figures. These precision columns therefore do not match the formats at equal effective precision. Resident VRAM is the per-process `nvidia-smi` figure after load for the stated configuration, not the file size.

results converge on—SC_2.00 with MTP dynamic drafting on one slot—is called the daily driver below; its final form is specified in Section 8, and rows that predate its last revision state their own configuration.

Library environment (E50). Every EXL3-lane measurement here (E7–E49, and the quant conversions of E32, E41, and E42) ran under a mixed CUDA library set: the lab’s shell exported `LD_LIBRARY_PATH=/usr/local/cuda-12.1/lib64`, so torch 2.10.0+cu128 loaded the system cuBLASLt and NVRTC 12.1 beside its bundled 12.8 libraries (read from `/proc/<pid>/maps`); the fault was found on 2026-09-10, after the campaign. Two pre-registered controls on 100 GSM8K items under a fully bundled set (H38d, E50; loaded libraries recorded per run) left the E49 non-reproducibility in place (64/100 undrafted same-config identity clean against 50/100 mixed) and the drafted wall time inside run-to-run variance (5.92s per item clean against 6.39 and 5.87s mixed), so the throughput rows stand as measured under the mixed set (Section 6).

Measurements, suites, and statistics. Timing boundaries differ by engine and must not be interpreted as matched kernel-only decode rates. Throughput is the server’s own `predicted_per_second` on llama.cpp and completion tokens over end-to-end wall time on TabbyAPI, which returns no timings (tokens counted through `/v1/token/encode`, E7). The EXL3 rate therefore includes prefill and request overhead; within-lane comparisons retain that same boundary. Timed generations use four prompt types (a math word problem, a code function with tests, a 400-word essay, a whole-file rewrite) at a 1,024-token cap, effort medium, temperature 1.0, with two repeats per prompt (E4, E7, E9) or three (E36, E44); resident VRAM is the per-process figure after load and after one warm request (E3, E27). Quality suites are the lab’s HumanEval+20 [18, 19], a 25-item MATH subset (math25, raw and lenient rescoring) [20], GSM8K-100 [21] (some historical rows are a 50-item sample), MMLU-200 [22], and a six-item needle suite (8K, 16K, 32K prompts, two positions), at a 4,096-token completion cap, effort medium, temperature 1.0, one run per item; a cap-truncated item without an answer scores wrong. Paired comparisons use the discordant items and the exact two-sided McNemar test [23]. One same-configuration repeat of GSM8K-100 moved from 98 to 95 correct (3 lost, 0 gained, $p = 0.25$; E26). This illustrates sampling variation; it does not estimate a general three-item noise band. The larger temperature-zero

(E50); it only widened the trigger of the converter’s existing CPU fallback, and its upstream pull request was closed (Section 8).

<i>Sustained single stream (E25): back-to-back math / code / prose / rewrite requests, 2,048-token cap</i>								
Lane	Minutes	Tokens	tok/s	Mean W	Idle W	tok/J gross	tok/J net	Capped
EXL3 SC_2.00 + MTP dyn., 12K	30.4	81,331	44.6	79.8	16.4	0.559	0.704	98%
UD-IQ2_S + MTP n3, 8K	15.1	41,437	45.8	79.7	17.6	0.577	0.740	96%
<i>Per prompt type (request means), tok/J EXL3 / IQ2_S: math 0.75 / 0.75; rewrite 0.73 / 0.75; code 0.64 / 0.65; prose 0.50 / 0.52</i>								
<i>Concurrency (E29): TabbyAPI, four slots, 12K, 5–6 min per cell, 1,024-token cap</i>								
Config	Clients	Aggregate	Per stream	Mean W	tok/J	vs 1 client		
EXL3 + MTP dyn.	1	43.0		43.7	79.6	0.54	—	
EXL3 + MTP dyn.	2	68.5	35.6 (−19%)		79.5	0.86	×1.59	
EXL3 + MTP dyn.	4	91.0	23.5 (−46%)		79.7	1.15	×2.12	
EXL3, no drafter	1	28.9		28.9	79.7	0.36	—	
EXL3, no drafter	4	75.6	19.1 (−34%)		79.7	0.95	×2.61	

Table 2: Tokens per joule at the 80 W cap. Energy is GPU-board energy integrated from 2s `nvidia-smi` samples; “Capped” is the share of all samples reporting the software power cap alone; the sustained tok/s includes prefill and inter-request gaps. Temperature 1, effort medium, loaded machine.

comparisons use 1,000 paired items for drafting and 200 for the higher-bit reference (Section 3.3). We report observed differences and paired tests, not statistical equivalence.

Pre-registration and evidence boundary. Predictions and falsifiers were recorded before each experiment block, starting on 2026-09-06 and ending with the release control on 2026-09-19. The artifact’s design record preserves the dated amendments, evidence commits, and subsequent corrections. Experiment labels E1–E52 identify those frozen records, rather than a proposed order of reading. The claim ledger distinguishes values recomputed from archived rows from values documented only in logs or the experiment record. Later literature and software changes inform the discussion but do not alter the frozen measurements.

3 Results

3.1 Throughput and tokens per joule

Drafting the EXL3 quant with its own MTP head raises completion throughput from 29 to 48.7–50.4 tok/s on code and math (1.67–1.74×) and to 35.7 on prose (1.2×), missing the pre-registered $\geq 1.8\times$ and $\geq 1.4\times$ (H1) by a small and a large margin (E7, four prompt types, means over two sessions; Table A1 gives every timed configuration). The same head as a sidecar gives 1.8× on the GGUF lane (E4b), and the external DFlash2 block drafter gives 2.0–2.6× on structured content but 1.2× on prose, where MTP is 15–18% faster (E4a; H2’s prose clauses falsified). Because the card sits at the cap in every row (79.2–79.8 W, E4a), tokens per joule are approximately throughput divided by 80 W; joules per token are power divided by throughput: drafting halves joules per token on structured content and saves 15–27% on prose (E4a). Decode is insensitive to the engine version ($< 1\%$ between llama.cpp builds, E1/E2; -0.2 to $+1.5\%$ between exllamav3 versions, E7) and to temperature (0.7 versus 1.0 moves acceptance by at most five points and throughput by 1–7%; E9); with depth it loses 15.2% from 2K to 24K on the GGUF lane (E20) and 22% from 2K to 30K on the EXL3 lane at 36K context, past H28’s 20% falsifier (E38).

The sustained runs (Table 2) deliver 0.559 and 0.577 tokens per joule gross (0.704 and 0.740 net of idle). The 3.2% difference is descriptive: one run per lane at unequal durations does not establish an energy ranking. Median SM clocks are 1,215 MHz on EXL3 and 1,380 MHz on llama.cpp; the cap was active in 96–98% of samples at 52–64 °C with no thermal-slowdown reason once generation started, the only drift a cold-start step inside minutes 0–5 on the EXL3 lane (46.6 to 44.2 tok/s; E25, H9 confirmed). Prose is the least efficient output per joule (0.50) because it drafts worst. Continuous batching turns the fixed cap into a multiplier (E29): two drafted streams reach 1.59×

the aggregate and 0.86 tok/J, four reach $2.12\times$ and 1.15 tok/J, and drafting keeps the lead at every batch size (91.0 versus 75.6 tok/s at four), although undrafted batching scales more steeply from a lower base; the per-stream loss at four drafted streams (-46%) exceeds H20’s 40% falsifier, as expected where a shared dynamic draft window makes every row pay verification for the longest window (engine reading; 24).

3.2 Draft acceptance, draft length, and memory allocation

Limited benefit from the tested verifier change. Acceptance is set by content—0.54 on prose, 0.70 on code, 0.73 on math, and 0.74 on rewrites over 9,821 drafted positions on the daily driver, decaying with draft position (0.77, 0.66, 0.53, 0.37 for positions 0–3; E35)—and short tool-call replies in an agent loop draft worse than long math (0.28–0.70 on llama.cpp, 0.50 on TabbyAPI), so decode there sits at 29–43 tok/s rather than 45–50 (E18, ten turns). Both engines draft the argmax of the MTP head and verify by sampling the target through the job’s sampler and accepting on equality, so per-position acceptance is $p(x^*)$, the target’s filtered probability of the drafted token; standard speculative sampling with a sampled draft $x \sim q$ [1, 2] accepts with probability $\sum_v \min(p(v), q(v))$ at the same output distribution, and an MTP-specific analysis attributes entropy-bounded acceptance to exactly this rule [25]. H32 predicted that the sampled rule would gain ≥ 0.10 on prose. Table A2 records the measurement-only patch’s per-set quantities: $p(x^*)$ predicts the observed acceptance within 0.007 on every set, but $\sum \min(p, q)$ sits within ± 0.035 of it everywhere, $+0.018$ on prose, because the MTP head is more confident than the target on its own argmax ($q(x^*)$ 0.69–0.83 against $p(x^*)$ 0.54–0.74) and puts the rest of its mass on tokens the target does not favor. H32 is falsified. The sampled-draft verifier was nevertheless implemented and A/B-tested (H33, E36): its small correctness checks passed (temperature-0 outputs byte-identical on three prompts—a check that predates the E49 finding of Section 3.3—and 300-completion histograms at temperature 1 consistent with the stock verifier) and reaches its theoretical acceptance (0.592 on 19,074 positions against $\sum \min(p, q) = 0.597$), but end-to-end speed fell on code (-5%), math (-8%), and rewrite (-3% ; medians of three repeats) and was unchanged on prose (34.3 to 33.9 tok/s). This verifier change did not improve throughput with the stock head. A different head or verification implementation remains an untested possibility, not a consequence ruled out by this experiment.

Draft length under dynamic drafting (H37, E44). The static cap `draft_num_tokens` still matters under dynamic drafting: across 2, 3, 4, and 6 the spread is 6% on prose, 7% on code, 8% on math, and 11% on rewrite (medians of three repeats; Table A3), but the pre-registered shape was wrong. Two is slowest on code, math and rewrite, while four is slowest on prose. Six is fastest on prose and math; three narrowly leads code (46.9 versus 46.8 tokens/s), and four leads rewrite. Six costs 256 MiB more VRAM than four. Dynamic sizing therefore does not eliminate the tradeoff between the configured cap, workload and memory.

The recurrent-state history tax and `max_batch_size` (H19, E27/E28). Enabling the drafter cost a fixed 2,432 MiB of process VRAM that moved neither with context nor with the draft cache’s precision (Table A4). Instrumented loads and a source reading agree on where it goes: for speculative rollback the main cache pre-allocates, per GDN layer, an fp32 tensor of shape (`max_batch_size`, `max_history`+1, 48, 128, 128)—3 MiB per slot and history step across 48 layers—and TabbyAPI sets `max_history` to the draft length (4) whenever a draft model exists and defaults `max_batch_size` to 4 for recurrent models: 4×5 states = 2,880 MiB drafted against 576 MiB undrafted, which with the MTP weights and draft cache predicts 2,437–2,453 MiB against 2,432 measured (E28). Each drafted slot costs 728 MiB at draft length 4 (438 at 2, 148 undrafted). For a single user, `max_batch_size: 1` reclaims 2,176 MiB (10,226 to 8,050 MiB at load) with de-

code within noise and byte-identical temperature-0 outputs on three 120-token prompts, and lets the drafted server load 32K context at 8,754 MiB and survive a 30K-token prompt (E28; H19 confirmed on every clause). The allocation follows the tested engine’s snapshot scheme; other rollback designs can change that cost (Section 7).

3.3 Paired accuracy and output reproducibility

The small temperature-1 drafted-versus-undrafted checks (E15, E17, E21; Table A5) did not resolve an accuracy decrease. A larger paired block therefore ran at temperature 0 on $n = 500$ supersets of the suites with each output hashed (8.0 h of GPU for H38a, 1.7 h for H38b, 52 min for H38c, plus the three 100-item runs of the E50 library-set control). Table A6 summarizes it and Tables A7–A9 give the full records; the predictions and falsifiers are restated in Appendix A (H38a: $\geq 95\%$ byte-identical text on 1,000 paired items with accuracy within 1.0 point at exact McNemar $p \geq 0.05$; H38b: within 2.0 points of UD-Q4_K_XL, llama.cpp with 30 of 64 layers on the GPU, on 200 GSM8K items at mean length within 15%). These historical decision rules are recorded faithfully, but a non-significant test plus a small point estimate is not an equivalence test; we do not interpret them as such.

E47 supported the accuracy clause on both suites, 97.6 against 97.2% on GSM8K-500 (0/2 discordant, $p = 0.50$) and 86.8 against 86.2% on MMLU-500 (6/9, $p = 0.61$), the drafted side nominally ahead with 11 of the 17 discordant items (pooled $p = 0.33$), and the temperature-1 speed-ups reproduced under greedy decoding ($1.79\times$ and $1.48\times$). The identity clause was falsified, and not narrowly: 63.6% and 21.0% byte-identical against the 90% falsifier. The differing pairs almost all agree on the answer (179 of 182 on GSM8K, 374 of the 387 parsed on MMLU), and identity falls with trace length as a constant per-token hazard, $P(\text{identical}) = (1 - \varepsilon)^L$ with $\varepsilon \approx 1.6 \times 10^{-3}$ and 2.6×10^{-3} : one greedy argmax flip every 400–600 tokens (Table A7). E48 supported H38b on both clauses: 99.0% for the 2-bit quant against 98.0% for the 4-bit GGUF (1/3 discordant, $p = 0.63$) at mean completion tokens of 322.0 against 321.9, so the +26% inflation of Section 3.5 does not appear in this GSM8K comparison under greedy decoding. The contrast suggests task- and sampling-dependent length effects; it does not identify their cause. The marker penalty was not tested on this larger paired block. Recorded, not an intrinsic speed comparison: 6.30 s per item for the resident 2-bit quant against 30.24 s for the offloaded 4-bit one.

Whether the hazard belonged to the drafter’s batched verify forward or to the engine was H38c (Table A9). A same-config re-run of the drafted daily driver on 100 GSM8K items, pre-registered at $\geq 97\%$ identity on the assumption that the engine is deterministic at batch 1, came out at 91/100, between the prediction and the $\leq 90\%$ falsifier; 25 paired greedy traces with top-2 log-probabilities per token put all five divergences at near-ties (undrafted top-1/top-2 gap at most 0.031 nat, the drafted token the runner-up in five of five), consistent with numerical near-ties rather than evidence specific to a verifier defect. Two undrafted runs of the same config, predicted at $\geq 97\%$ identity in a control added before it ran, agreed on 50/100, past the $\leq 90\%$ falsifier. The engine’s own single-token greedy decode is thus not bit-reproducible on this model under exllamav3 1.4.7, with a per-token hazard of 2.5×10^{-3} , about seven times the drafted path’s 3.4×10^{-4} ; one flip changes the rest of the trace and almost never the answer (50 of 50 differing pairs). Because those runs used the mixed library set of Section 2, H38d re-ran the undrafted control under a fully bundled set, pre-registered on 2026-09-10 before it ran at $\geq 97\%$ identity if the non-determinism were a library artifact ($\leq 60\%$ the falsifier): it measured 64/100 [54.2, 72.7] (E50), inside the interval of the mixed 50/100, clean run 1 agreeing with the two mixed runs on 58 and 61 items, and drafted clean against drafted mixed at 91/100, the same as mixed against mixed. Thus engine-level variation confounds the drafted-versus-undrafted identity comparison under either library set. The supported accuracy statement is that drafting showed no measured decrease on these 1,000 items.

The higher-bit comparison likewise records 99.0 versus 98.0% and nearly equal mean length on 200 GSM8K items, without proving equivalence. Run-to-run numerical nondeterminism is known in LLM inference [26]; these controls show why an identity test on the measured build needs an undrafted same-configuration reference.

Following an upstream report on 2026-09-10, the maintainer added a deterministic reduction in the Gated-DeltaNet kernel on exllamav3’s development branch (commit 900cf17, built and tested here at the branch head 08849e3, after the 1.4.9 release). Built here against the pinned torch and run under the same clean library set, two undrafted greedy runs on the same 100 items were byte-identical on 100 of 100 with accuracy 97 and 97 (E51; predicted at $\geq 97\%$ before the run); against 1.4.7 the same prompts agree on 67/100 at equal accuracy, the expected signature of a changed reduction order. The fix shipped in the released exllamav3 1.5.0: its official wheel, served unmodified through the same pinned environment, again gave two byte-identical greedy runs on 100 of 100 items, accuracy 98 and 98 (E52; pre-registered as H38f). The non-identity of E47, E49 and E50 is therefore a property of the measured 1.4.7 build, and the identity clause of H38a becomes a meaningful test on the tested 1.5.0 configuration. The 100-item repeat does not establish determinism for every input, batch shape or later release.

3.4 Prefill anatomy and the checkpoint interval

Prefill is slower on the measured EXL3 path and can dominate an edited-prompt request. Profiled on the bare API for one 3,652-token prefill after warm-up (E37; 1,515 MHz at 79.8 W), GPU time splits 73.8% cuBLAS fp16 GEMM, 10.7% trellis-to-fp16 reconstruction, 4.8% copies, 3.5% Gated-DeltaNet chunk kernels, 2.9% paged attention, and about 4% norms and miscellany, at 583 tok/s with a 1,024-token chunk (535 at 512, 592 at 2,048). Every chunk rebuilds the whole 2-bit body as fp16 (about 0.24s per 1,024-token chunk, no cache mode) and multiplies it with fp32 accumulation at 37 TFLOPS against a 43.7 TFLOPS FLOP-weighted ceiling for these shapes. Doubling the default chunk improves this measurement by only 1.5%, although 512 to 2,048 improves it by 10.7%. The profile identifies where time is spent; it does not rule out other kernels, caching strategies or allocation changes. The GGUF lane uses a different integer tile-dequant path and prefills at 600–710 tok/s (E20) and holds 686–690 tok/s at 8K and 609–618 at 32K with matched KV types (E5), making it faster on these measured prefills. On the GGUF lane the prefill collapse the lab had recorded at 32K is a configuration artifact: the mismatched pair `-ctk q8_0 -ctv q4_0` runs attention off the fused kernels at 13–15% GPU utilization, 57 tok/s at 8K and past a 600-s deadline at 32K (E5, one needle item per cell; H4 confirmed), and the needle suite goes from 4/6 to 6/6 with 32K items answered in 49 s (E16a).

Two engine behaviors set the latency an agent sees. A repeated prefix is cheap on both lanes because the recurrent state is checkpointed: append-only turns of a synthetic twelve-turn agent conversation cost a flat 1.8–2.0s of prefill per roughly 1.1K new tokens on llama.cpp with default checkpoint settings (E6; H5’s failure prediction did not materialize) and 2.0–2.6s on TabbyAPI (E18b2), and a cold 4K system prefix that took 9.3–9.4s is served in 0.9s on its second use and still after an unrelated conversation, from a 148 MiB host-RAM checkpoint—with room for one checkpoint the reuse is lost (E30, two repeats; H21 confirmed). An edited prompt, by contrast, replays from the last checkpoint at or before the edit, never from the edit: an edited header at turn six costs one full re-prefill (13.0–13.1s at about 7.8K tokens) on both engines (E18), and because TabbyAPI’s ingestion checkpoints are sparse by default, any edit before token 4,096 of a 5,237-token document costs the full 9.9s and only an edit after it costs 3.0s (E38; H29’s mechanism confirmed, its predicted positions falsified). Table A11 shows the lab patch that exposes the interval: at 1,024 tokens an edit 2,200 tokens from the end costs 5.0s instead of 10.1s and the cost becomes proportional to the distance from the edit to the end of the prompt, for +2–3% on the cold prefill

and 148 MiB of host RAM per extra checkpoint (E43; H35 confirmed). The pinned TabbyAPI checkout does not expose the knob; the patch is two lines of engine code and one passthrough, and the passthrough was merged into TabbyAPI after these measurements (pull request #468, 2026-09-21).

3.5 Two-bit reasoning length: the inflation, its mechanism, and the levers

The tested lower-bit artifacts produce longer outputs on math25: on matched items the median completion is 861 tokens at UD-Q4_K_XL, 1,083 for the drafted EXL3 2.00 (+26%), 1,381–1,447 for UD-IQ2_S (+60–68%), and 1,466 for UD-IQ2_XXS (+70%) (E8, $n = 25$, temperature 1; H14 confirmed), with no inflation on GSM8K (409–434 mean tokens across tiers, $n = 50$; R0). Table A10 examines the output differences using temperature-0 traces and per-token log-probabilities on the same 25 math and 20 code prompts (E40). The extra length is entirely in the math reasoning and it is re-verification: $2.4\times$ the “Wait”-class transitions per trace and explicit self-corrections up from 1 to 4, while code does not inflate ($0.98\times$). The second half of the pre-registered mechanism came out the opposite of what was predicted: the 2-bit model is not less sure at those points—its greedy tokens fall below $p = 0.5$ less often (1.4% versus 2.3%), the transition tokens sit at a similar median probability (0.84 versus 0.80), and the traces already differ at the third token (median two matching tokens on 24 of 25 math items)—so nothing in per-token confidence marks the branch; final answers agree on 22 of 25 items. H26’s transition clause held and its low-confidence, divergence, and corrections clauses were falsified. The reading that survives—the quantization shifts the model’s policy toward more checking on hard math, a confident behavioral drift rather than noise at high-entropy branch points—is an inference from these counts, not a measured cause.

The levers. Table A12 collects every intervention the campaign tried against the tax, each against its pre-registered prediction. Three cheap levers on the quantization itself are nulls. Recalibrating the stock per-tensor recipe on 155 task-focused reasoning traces the 2-bit model wrote itself (166K tokens, filled to the converter’s 250 rows with turboderp’s own trace) moved completion tokens within $\pm 6\%$, with no statistically resolved accuracy change on the three small suites (E32; H23 falsified). The stock quant already uses self-generated reasoning calibration [27, 28]. Moving bits at a constant 2.00 bpw from the lowest-error 3-bit and 4-bit projections to the six worst 1-bit MLP tensors cut those tensors’ proxy error fivefold, yet by layer 50 the two error measures were similar (0.08490 versus 0.08457), without the predicted downstream token reduction (E41; H25 falsified): the output-side error re-equilibrates within a dozen layers, in line with the “damage is diffuse” finding for other models [29]. The sampled-draft verifier is the third null (E36). The control that turns the calibration null into a result is off-policy calibration (H31, E42): the same recipe converted on 250 rows of generic prose and standard-library Python raised GSM8K-100 completion tokens by 15.6% and math25 by 6.3%, quintupled math25 cap truncations (1 to 5), and lost items on every suite; no single suite reaches significance, but the discordant items run 9 to 1 for the self-calibrated quant across the three suites (exact binomial $p = 0.021$). The off-policy recipe worsened these measurements; the tested on-policy recipe did not yield the predicted improvement. This does not exhaust possible calibration choices.

Of the decoding-time levers, the overthinking-marker logit penalty [30] is a lever on the GGUF lanes (math25 median token ratios at -2.0 : 0.85 on UD-IQ2_XXS, 0.94 on UD-IQ2_S, 1.03 on EXL3; E13, E19) and a small observed saving on the trellis lane, where a strength sweep (E45) found the accuracy clause holding at every strength (0–2 items, all $p \geq 0.5$) but the saving flat at about 5% from -0.5 to -3.0 and largest at the weakest setting (H36’s monotonicity clause falsified): half a nat is enough to route around the marker token, and the model then re-verifies in other words. A client-side trial-answer early exit [31] saved 18–24% of GSM8K-100 tokens and 6–20% of math25

tokens at an accuracy cost equal in size to the change in one baseline repeat, but not thereby established as harmless (E24, E26); forcing an answer at 90% of the budget produced 0 correct answers on the 4 items it fired on (E31; H22 falsified), consistent with the report that an injected `</think>` does not end reasoning [32]; lowering `reasoning_effort` to low collapsed HumanEval+ while taking longer on it (E14; H10 falsified in the opposite direction); and the 2.20-bpw branch buys nothing measurable at 608 MiB more VRAM (E23c; H18 confirmed).

3.6 Shared serving for two or three agents

A box that runs several agents can serve them from one drafted TabbyAPI, but the recurrent-history slots price the choice (Table A13; E39: synthetic agents with 3.7K-token system prompts and six append-only 700-token turns, about 8.2K tokens by the last). The pre-registered three-slot, 16K configuration fails: three multi-turn agents need about 24K tokens of KV by their last turns, so a 16K cache evicts and re-prefills continuously, later-turn time to first token climbs to 27.6s, and the process reaches 10.91 GiB (H27 falsified). With two slots and a 24K cache two agents each keep 34.3 tok/s (−33% versus 51.3 alone) at 3.1s later-turn TTFT, but the aggregate gain is $1.31\times$, under the $1.59\times$ of continuous batching (E29), because each new turn’s prefill stalls the other agent’s decode and the agents idle between turns; a third agent queues and pushes later-turn TTFT to 13.7s at no aggregate gain. A two-slot drafted server at 24K is the better of these tested shared configurations (`serving/tabby-2agents.yml`). More clients queue in this configuration. An undrafted server uses 148 rather than 728 MiB per slot, but its single-stream rate is lower (28.9 versus 43.0 tok/s, E29); its suitability for a particular multi-agent workload requires a separate test.

4 Negative Results

The detailed prediction ledger is in Table A14. Several plausible changes did not deliver their predicted improvement: the sampled-draft verifier was no faster; one on-policy calibration and one constant-bit reallocation did not substantially shorten outputs; forced early answers failed on all four affected math items; and four-way batching raised aggregate throughput while reducing each stream’s speed by 46%. These are bounded negative results for named implementations and samples, not evidence that the broader approaches cannot work. The small accuracy comparisons remain inconclusive about modest gains or harms.

5 Discussion

For a single user, the clearest configuration gain is reducing unused concurrency allocation. The recurrent-history tensor’s shape predicts the observed drafting overhead, and changing the slot count lowers load memory by 2,176 MiB. This preserves space for context without changing the weights. For repeated requests that edit cached text, denser recurrent checkpoints offer a separate latency–host-memory tradeoff. These are actionable serving changes, not new inference algorithms.

The remaining diagnostics describe the tested implementation. With the stock MTP head, replacing argmax proposals with the tested sampled rule offers little acceptance gain and no measured speed gain. In the profiled EXL3 prefill, fp16 matrix multiplication dominates; the modest improvement from a larger chunk does not rule out a different kernel or a separate prefill representation. Recent disaggregated quantization explicitly explores that alternative [33]. Neither result justifies declaring a permanent limit of the model or format.

The longer math traces require more caution. We observe extra tokens and re-verification markers, but the comparison changes artifact and engine together. A shift toward repeated reasoning is a plausible interpretation, not an isolated causal effect of quantization. The tested recalibration and bit-allocation variants do not settle what another recipe, training method or reference precision

could achieve.

At a binding power cap, improving throughput tends to improve tokens per GPU-board joule by a similar factor. This does not make tokens/J a quality metric: a longer wrong answer can score well on throughput, and board telemetry excludes host energy. Batching improves aggregate tokens/J here while slowing individual replies. The choice between a single-user and a shared configuration therefore depends on latency and concurrency needs, not one universal efficiency ranking.

6 Limitations and Claim Ceiling

The accuracy results are observations on finite samples, not demonstrations of equivalence. E47 tests 500 GSM8K and 500 MMLU items; the drafted point estimates are slightly higher, but non-significant McNemar tests cannot establish losslessness. E48 compares two bits with the higher-bit GGUF only on 200 GSM8K items. Larger MMLU or MATH comparisons against that reference were not run. The smaller temperature-1 evaluations use 20–200 items with one sample per condition; one baseline repeat differs by three GSM8K answers, which cannot define a general noise threshold. Small paired effects remain uncertain [34, 35], and similar aggregate accuracy can conceal different wrong answers [36].

All results concern one model, one GPU, one 80 W cap and the named artifacts and engines. The host carried other workloads. Timed prompt tests use only two or three repeats, and the sustained energy comparison has one run per lane at different durations. Several quality comparisons use earlier-day baselines; one uses 50 rather than 100 GSM8K items. Mechanism probes cover one profiled prefill or small trace sets. Energy is GPU-board energy, not whole-system energy or energy per correct answer. Cross-engine comparisons cannot isolate a quantizer’s causal effect; nominal bit labels are not matched effective precision. The faster resident two-bit run versus an offloaded higher-bit reference does not establish an intrinsic engine ranking.

EXL3 experiments E7–E49 loaded system cuBLASLt and NVRTC 12.1 alongside torch’s bundled 12.8 libraries. Fully bundled controls retained the nondeterminism (64/100 identical undrafted pairs versus 50/100 with mixed libraries), and the drafted timing lay within the observed run-to-run spread. These limited controls do not show that every earlier throughput or quality result is unaffected; the main sustained runs were not repeated under the corrected environment.

Most performance evidence uses exllamav3 1.4.7. Later controls test output identity, not a complete performance rerun. By the September 28 literature check, exllamav3 1.5.2 was available and TabbyAPI exposed the checkpoint options studied here. New engines, draft heads and quantized checkpoints are not ranked by this report. The recommended files preserve a measured historical configuration, not a claim to the best current local model. The three experimental re-calibration/recipe weight sets were deleted; their hashes and conversion recipes remain, but exact reruns require regenerating them.

7 Related Work

Direct deployment comparisons. MTP acceleration of Qwen3.8-27B is already documented. Ollman’s author-maintained RTX 3090 report measures EXL3 at 3.0 bpw with decode increasing from 43.6 to 94.8 tokens/s, alongside long-context fidelity tests [4]. Ang’s first-party RTX 3090 study measures GGUF drafting speed and GPU energy at about four bits, plus a separate lower-bit quality and memory ladder; it explicitly leaves board-power measurements below four bits untested [5]. Tsai provides another desktop MTP benchmark [6]. These are useful deployment reports, not independent replications of our laptop tests. Their different GPUs, power, precision, workloads and software prevent a normalized speed or quality ranking. Our contribution is the paired low-bit evidence and diagnostic detail at the 12 GB, 80 W operating point, not the discovery that this model can self-draft.

Quantization and reasoning length. Longer reasoning under low-bit quantization is already reported for other models and scalar quantizers [30, 37–39]. For this model, Kozyrev and Maiboroda report no inflation with a particular four-bit NVFP4 intervention [40], while a community GGUF benchmark reports longer two-bit outputs [41]. The KTopt author model card also measures reasoning lengths for a roughly 3.06-bpw trellis GGUF with an optional correction adapter against Q8 [42]. Thus neither trellis quantization nor its effect on length is an unmeasured category. Our 2.00-bpw EXL3 results describe a different artifact, engine and task sample.

Lotfi et al.’s marker penalty motivates our decoding-time test [30]. Self-generated calibration and task-aware calibration are established approaches [27, 28, 43]; our null results concern specific changes to an already self-calibrated checkpoint. The bit-allocation probe is consistent with distributed quantization damage [29], but does not prove its mechanism. Panferov et al. train separate compute-native prefill weights and evaluate SSD-offloaded prefill for Qwen3.8-27B [33]. That work is an alternative to the pinned weight-only path profiled here, not something our chunk-size experiment rules out.

Drafting and recurrent state. Exact speculative verification predates this report [1, 2]; BE-BOP analyzes MTP rejection sampling with a head trained for that rule [25]. We test the stock head and one verifier implementation, not their method under matched conditions. DFlash supplies an alternative block drafter [13]. SpecLA, TreeWY and Bole reduce recurrent-state verification costs through factor or replay designs [44–46], and GDN Tree-Scan serves branch-local replay in vLLM [47]. Our slot measurement concerns the snapshot design in the pinned deployment. Recurrent-state precision is itself an active optimization target [9]; batching and speculation also interact [24].

Energy and reproducibility. Consumer-GPU energy and speculative-decoding energy are already benchmarked [3, 48]. Ultra-low-bit work also measures a 2B model on Intel laptop GPUs [49]. Luo et al. profile this 27B model at BF16 under much higher workstation power caps and show a substantial gap between GPU-board and whole-system energy [50]. We report only board energy and do not compare those figures as hardware efficiency rankings. Numerical nondeterminism is also known [26]; our contribution is a reproducible engine-specific report and the bounded control tests of its upstream fix.

8 Artifact Availability and Reproducibility

The version-1.0.0 reproducibility bundle uses DOI [doi:10.5281/zenodo.22847652](https://doi.org/10.5281/zenodo.22847652) and the versioned source path [research-artifacts](#), tag [drafted-2bit-local-efficiency-v1.0.0](#). The DOI identifies the frozen evidence bundle, not a separate manuscript deposit. Public availability is established by the published Zenodo record and matching GitHub tag, not merely by the presence of these identifiers.

Analysis-only reproduction uses Python 3.11 or newer:

```
python3 check_claims.py
```

It checks the exported data hashes and recomputes the principal numerical claims without a model, GPU or network access. `CLAIM_EVIDENCE_MAP.md` maps each claim to archived inputs and identifies figures supported only by log extracts or the experiment record. `DESIGN_RECORD.md` records the frozen experimental design and revisions.

Full inference replication is separate. `FULL_RERUN.md` names exact engine and model revisions, hashes, patch application checks, commands and expected outputs. The bundle includes serving configurations, evaluation and measurement tools, sanitized patches, item-level outcomes, numeric trace summaries and environment metadata. It excludes model weights, benchmark text and

model-response text. The original generated patch excerpts are provenance records where identified as such; the rerun guide names the complete applicable patch. Replication requires compatible hardware and substantial model execution, and does not promise identical historical timings.

The checkpoint-interval interface was merged upstream in [TabbyAPI #468](#) on September 21. The slot-allocation logging proposal [#467](#) remained open on September 28. The nondeterminism report [exllamav3 #353](#) led to the maintainer’s deterministic reduction, tested here in development and release 1.5.0 (E51/E52). The separate [Cholesky fallback proposal #352](#) was withdrawn after the local library-path fault was found; it is not a claimed engine fix. We thank turboderp for the diagnostic question that exposed that fault and for the reduction change.

9 Conclusion

Self-drafting makes the tested two-bit Qwen3.8-27B deployment faster within a 12 GB, 80 W laptop budget: EXL3 completion throughput rises from about 29 to 48.7–50.4 tokens/s on code and math and 35.7 on prose. Its sustained GPU-board efficiency is 0.559 tokens/J, compared with 0.577 for the tested drafted GGUF run. Reducing unused concurrency slots saves 2,176 MiB, and a denser checkpoint grid reduces one edited-prompt latency from 10.1 to 5.0 s at a host-memory and cold-prefill cost.

The paired accuracy tests show no measured decrease with drafting on 1,000 greedy items, not equivalence or losslessness. Prefill profiling, acceptance measurements and output-reproducibility controls explain limits of the pinned implementation and identify changes worth testing. They do not establish permanent architectural limits or rank newer engines. The usable result is an auditable deployment baseline, practical configuration changes, and a clear record of where additional comparisons are needed.

AI assistance. Claude Code (Anthropic) operated experiments under the author’s standing rules and pre-registrations; wrote engine patches, measurement tools and analysis scripts; drafted the experiment record, manuscript, claim ledger and bibliography; and prepared the artifact exporter and claim checker. Codex (OpenAI) reviewed the evidence and prose, checked primary-source literature and upstream status, revised the manuscript and companions, and checked the reading builds and artifact instructions. Matthew Schwartz directed the research and is responsible for its methods, results, interpretation, claims, citations, rights, and released artifacts.

References

- [1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2022. [arXiv:2211.17192](#).
- [2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. [arXiv:2302.01318](#).
- [3] Rohit Dutta, Paramita Koley, Soham Poddar, Janardan Misra, Sanjay Podder, Naveen Balani, Saptarshi Ghosh, and Niloy Ganguly. Benchmarking the energy savings with speculative decoding strategies, 2026. [arXiv:2602.09113](#).
- [4] Alexander Ollman. Qwen3.8-27B on RTX 3090: Results. Author-maintained technical benchmark report, 2026. URL <https://alexander-ollman.github.io/qwen3.8-on-rtx3090/results.html>. Primary report, not peer-reviewed; accessed 2026-09-28.
- [5] Chin Keong Ang. Qwen3.8-27B on a 24 GB RTX 3090: Measured settings, speeds and energy. Author-maintained technical benchmark report, 2026. URL <https://chinkeong.github.io/qwen-27b/index.html>. Measurements dated 2026-08-21 through 2026-08-25; not peer-reviewed; accessed 2026-09-28.
- [6] Hsiu-Chi Tsai. Qwen3.8-27B speculative decoding on a single RTX 3090. Zenodo software deposit, version 1.0.4, 2026. URL <https://doi.org/10.5281/zenodo.22246518>. Deposited 2026-09-02; concept DOI 10.5281/zenodo.22149941; source <https://github.com/thc1006/qwen3.8-speculative-decoding-rtx3090>; accessed 2026-09-19.
- [7] Qwen Team. Qwen3.8-27B model card. Hugging Face model repository, 2026. URL <https://huggingface.co/Qwen/Qwen3.8-27B>. Accessed 2026-09-06 (lab community sweep).
- [8] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule, 2024. [arXiv:2412.06464](#).
- [9] Tao Zhang, Jianchao Tan, Pingwei Sun, Yanqi Yu, Zixu Jiang, Yuchen Xie, Xunliang Cai, and Ziqian Zeng. DAMP: Decay-aware mixed-precision recurrent-state quantization, 2026. [arXiv:2608.27513](#).

- [10] turboderp. Qwen3.8-27B-exl3, branch SC_2.00bpw_H3. Hugging Face model repository, 2026. URL https://huggingface.co/turboderp/Qwen3.8-27B-exl3/tree/SC_2.00bpw_H3. Shard SHA-256 digests verified against the Hub LFS records on 2026-09-06 (hash_audit_2026-09-06.json); the SC_2.20bpw_H3_V3 branch was verified the same way.
- [11] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. QTIP: Quantization with trellises and incoherence processing, 2024. [arXiv:2406.11235](https://arxiv.org/abs/2406.11235).
- [12] Unsloth. Qwen3.8-27B dynamic GGUF quantizations. Hugging Face model repository, 2026. URL <https://huggingface.co/unsloth/Qwen3.8-27B-GGUF>. Files UD-IQ2_S, UD-IQ2_XXS, UD-Q4_K_XL, mmproj-F16 and MTP/mtq-Qwen3.8-27B-Q4_0 as re-cut on 2026-08-20 (Dynamic v3.0); SHA-256 digests verified 2026-09-06.
- [13] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding, 2026. [arXiv:2602.06036](https://arxiv.org/abs/2602.06036).
- [14] z-lab. Qwen3.8-27B-DFlash2-GGUF. Hugging Face model repository, 2026. URL <https://huggingface.co/z-lab/Qwen3.8-27B-DFlash2-GGUF>. Q4_K_M drafter file; SHA-256 digest verified 2026-09-06.
- [15] theroyallab. TabbyAPI. Software repository, 2026. URL <https://github.com/theroyallab/tabbyAPI>. Pinned lab checkout, commit 4a4f9f4 (2026-08-24); pull request 468 merged 2026-09-21 as commit 0feab2a; accessed 2026-09-25.
- [16] turboderp-org. exllamav3. Software repository, versions 1.4.7 and 1.5.0, 2026. URL <https://github.com/turboderp-org/exllamav3>. Version 1.4.7 (released 2026-09-05), staged in isolation from the lab’s pinned 1.4.3; development-branch commit 08849e3 (2026-09-12; contains the deterministic Gated-DeltaNet reduction 900cf17) for E51; release 1.5.0 (2026-09-13) for E52; accessed 2026-09-13.
- [17] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. Software repository, 2023. URL <https://github.com/ggml-org/llama.cpp>. Release v0.4.0 (commit 5266f24) and the 2026-08-26 build (commit 035e227) were used; accessed 2026-09-06.
- [18] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. [arXiv:2107.03374](https://arxiv.org/abs/2107.03374).
- [19] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation, 2023. [arXiv:2305.01210](https://arxiv.org/abs/2305.01210).
- [20] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset, 2021. [arXiv:2103.03874](https://arxiv.org/abs/2103.03874).
- [21] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. [arXiv:2110.14168](https://arxiv.org/abs/2110.14168).
- [22] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2020. [arXiv:2009.03300](https://arxiv.org/abs/2009.03300).
- [23] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996. URL <https://doi.org/10.1007/BF02295996>.
- [24] Qidong Su, Christina Giannoula, and Gennady Pekhimenko. The synergy of speculative decoding and batching in serving large language models, 2023. [arXiv:2310.18813](https://arxiv.org/abs/2310.18813).
- [25] Yucheng Li, Huiqiang Jiang, Yang Xu, Jianxin Yang, Yi Zhang, Yizhong Cao, Yuhao Shen, Fan Zhou, Rui Men, Jianwei Zhang, An Yang, Bowen Yu, Bo Zheng, Fei Huang, Junyang Lin, Dayiheng Liu, and Jingren Zhou. Breaking entropy bounds: Accelerating RL training via MTP with rejection sampling, 2026. [arXiv:2606.12370](https://arxiv.org/abs/2606.12370).
- [26] Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference, 2025. NeurIPS 2025 (oral). [arXiv:2506.09501](https://arxiv.org/abs/2506.09501).
- [27] Miles Williams, George Chrysostomou, and Nikolaos Aletras. Self-calibration for language model quantization and pruning, 2024. [arXiv:2410.17170](https://arxiv.org/abs/2410.17170).
- [28] Shigeng Wang, Chao Li, Yangyuxuan Kang, Jiawei Fan, and Anbang Yao. Attend to your own thoughts: Breaking the barrier for post-training quantization of reasoning LLMs through the lens of 1.58-bit quantization, 2026. [arXiv:2608.01078](https://arxiv.org/abs/2608.01078).
- [29] Jundong Hu and Shekar Ramachandran. The structure of quantization damage in LLMs: Why the next bit should be spent globally, 2026. [arXiv:2609.01587](https://arxiv.org/abs/2609.01587).
- [30] Sanae Lotfi, Polina Kirichenko, Steven Li, and Zechun Liu. Quantized reasoning models think they need to think longer, but they do not, 2026. [arXiv:2606.00206](https://arxiv.org/abs/2606.00206).
- [31] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models, 2025. [arXiv:2504.15895](https://arxiv.org/abs/2504.15895).
- [32] Seunghye Koh, Sungjae Choi, Minchan Kwon, Sunghyun Baek, and Junmo Kim. </think> doesn’t stop reasoning: Analysis of spurious CoT termination, 2026. [arXiv:2609.03633](https://arxiv.org/abs/2609.03633).
- [33] Andrei Panferov, Maximilian Kleinegger, Sweta Priyadarshi, Tijmen Blankevoort, and Dan Alistarh. Disaggregated quantization: Specializing LLM prefill and decode, 2026. arXiv 2609.26333v1, submitted 2026-09-22.
- [34] Andreas Hochlehnert, Hardik Bhatnagar, Vishaal Udandarao, Samuel Albanie, Ameya Prabhu, and Matthias Bethge. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility, 2025. [arXiv:2504.07086](https://arxiv.org/abs/2504.07086).
- [35] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. [arXiv:2411.00640](https://arxiv.org/abs/2411.00640).

- [36] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. Accuracy is not all you need, 2024. [arXiv:2407.09141](#).
- [37] Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization hurts reasoning? An empirical study on quantized reasoning models, 2025. [arXiv:2504.04823](#).
- [38] Xinyu Lian, Walid Krichene, Beichen Huang, Masahiro Tanaka, Olatunji Ruwase, Li Zhang, and Minjia Zhang. Quantization inflates reasoning: Token inflation as a hidden cost of low-bit reasoning models, 2026. [arXiv:2606.25519](#).
- [39] Ekaterina Alimaskina, Darya Rudas, Denis Shveykin, Gleb Molodtsov, Pavel Vasiliev, and Aleksandr Beznosikov. Extreme low-bit inference in reasoning models: Failure modes and targeted recovery, 2026. [arXiv:2606.02011](#).
- [40] Sergii Kozyrev and Davyd Maiboroda. Why gated DeltaNet survives 4-bit quantization: NVFP4 W4A4 for the recurrent half of a hybrid 27B LLM, 2026. [arXiv:2609.04098](#).
- [41] Piotr Migdał. Benchmarking Qwen3.8 27B quantizations: 4-bit holds up, 1-bit collapses. Quesma blog, 2026. URL <https://quesma.com/blog/qwen38-27b-quantizations-benchmarked/>. Published 2026-08-26; accessed 2026-09-19.
- [42] wiklif. Qwen3.8-27B-KTOpt-GGUF: Model card and benchmarks. Author-maintained Hugging Face model card, 2026. URL <https://huggingface.co/wiklif/Qwen3.8-27B-KTOpt-GGUF>. First-party artifact measurements, not peer-reviewed; accessed 2026-09-28.
- [43] Fei Wang, Chao Xue, Taoran Liu, Li Shen, Ye Liu, and ChangXing Ding. Beyond activation alignment: The alignment-diversity tradeoff in task-aware LLM quantization, 2026. [arXiv:2607.00908](#).
- [44] Zhibin Wang, Xuying Han, Zhaohua Yang, Fuliang Liu, Xue Li, Rong Gu, Sheng Zhong, and Chen Tian. SpecLA: Efficient speculative decoding for linear-attention models, 2026. [arXiv:2607.16673](#).
- [45] Sneha Murthy Ghantasala. TreeWY: Speculative verification for gated DeltaNet hybrids, 2026. [arXiv:2608.20961](#).
- [46] Li Wang, Yi Su, Xiabao Wu, Chiran You, Yongchao Liu, Zhan Qiu, Juelu Zhang, Jiajun Zheng, Fangxin Liu, Jie Zhang, Chen Tian, and Chengying Huan. Bole: Efficient tree speculation for hybrid-attention language models, 2026. [arXiv:2608.01651](#).
- [47] Zhiyuan Ma. GDN tree-scan: Served tree verification for recurrent-hybrid language models, 2026. [arXiv 2609.23900v1](#), submitted 2026-09-20.
- [48] Philipp M. Zähl and Anika Hennig. Energy efficiency of locally deployed LLMs: A preliminary quantitative GPU power benchmark on consumer hardware, 2026. [arXiv:2608.00008](#).
- [49] Evangelos Georganas, Dhiraj Kalamkar, Alexander Heinecke, and Pradeep Dubey. Pushing the envelope of LLM inference with ultra-low-bit quantized models, 2025. [arXiv 2508.06753v3](#), 2026-08-27.
- [50] Qi Luo, Kunlin Li, Ziwen Wang, and Yun Chen. Where does the energy go? profiling LLM agent inference on Blackwell GPUs, 2026. [arXiv 2609.29707v1](#), submitted 2026-08-31.

A Detail Tables

These tables carry the full records behind Sections 3 and 4; the main text keeps its compact tables where the page budget allows. Table A1: every timed configuration (Section 3.1). Table A2: the E35 acceptance quantities; Tables A3 and A4: draft length and resident VRAM (Section 3.2). Table A5: the temperature-1 quality checks that the hardening block superseded (Section 3.2). Table A6: the hardening block in summary, and Tables A7–A9 its full records including the E50 library-set control (Section 3.3). Table A10: the E40 trace comparison (Section 3.5). Table A11: the checkpoint-interval probe (Section 3.4). Table A12: every lever against the reasoning-length tax (Section 3.5). Table A13: shared serving (Section 3.6). Table A14: every falsified or missed pre-registered prediction (Section 4).

Pre-registered predictions and falsifiers of the hardening block. H38a (01:40 on 2026-09-09, before any GPU use): byte-identical reasoning-plus-answer text on $\geq 95\%$ of 1,000 paired greedy items with accuracy within 1.0 point per suite at exact McNemar $p \geq 0.05$; identity in 90–95% with accuracy inside the band pre-registered as “accuracy-lossless but not sample-identical”; falsifiers identity below 90% on either suite, $|\Delta| > 1.5$ points, or $p < 0.05$ with the undrafted side ahead. H38b (same time): the 2.00-bpw quant within 2.0 points of UD-Q4_K_XL at $p \geq 0.05$ on 200 paired GSM8K items with mean completion tokens within 15%; falsifier the 4-bit quant ahead by more than 3 points at $p < 0.05$; the wall-clock ratio recorded, not claimed, because the offload split is arbitrary. H38c ((i) and (ii) at 05:30 on 2026-09-09 after an early H38a read, (iii) at 14:25 after (i) had been read and before it ran): (i) a drafted same-config re-run at $\geq 97\%$ identity, falsifier $\leq 90\%$; (ii) at the first divergent token of 25 paired traces, the undrafted top-1/top-2 gap below 0.5 nat and the drafted token the runner-up in $\geq 80\%$ of divergences, with the median first divergence beyond 100 tokens, falsifiers gaps ≥ 1 nat or drafted tokens outside the top two; (iii) an undrafted same-config re-run at $\geq 97\%$, falsifier $\leq 90\%$. H38d (11:45 on 2026-09-10, before it ran): undrafted same-config identity $\geq 97\%$ under a fully bundled CUDA library set, falsifier $\leq 60\%$; its speed addendum (12:30, before it ran): drafted wall per item within $\pm 5\%$ of the mixed-set 6.49 s (E47) and identity against the mixed drafted run in 55–90%. H38e (04:10 on 2026-09-12, before it ran): two undrafted greedy runs on exllamav3’s development branch byte-identical on $\geq 97\%$ of the same 100 items with accuracy unchanged, falsifier $\leq 75\%$. H38f (08:43 on 2026-09-19, before it ran): the same test on the unmodified release wheel of exllamav3 1.5.0, identity $\geq 97\%$ and accuracy within two items of E51’s 97, falsifier $\leq 75\%$.

Target (context)	Drafter	Code	Math	Prose	Rewrite	J/tok code / prose
EXL3 SC_2.00 (36,864)	none	29.2	29.0	29.3	28.6	2.73 / 2.72
EXL3 SC_2.00 (16,384)	MTP, dynamic	48.7	50.4	35.7	49.5	1.64 / 2.24
EXL3 SC_2.00 (8,192)	MTP, 3 tokens	47.5 (0.58)	49.6	34.7 (0.33)	49.5 (0.65)	1.68 / 2.30
UD-IQ2_S (8,192)	none	23.9	23.9	24.0	23.9	3.36 / 3.35
UD-IQ2_S (8,192)	MTP sidecar, n-max 3	42.5 (0.75)	45.0 (0.81)	31.6 (0.47)	48.5 (0.91)	1.90 / 2.54
UD-IQ2_S (4,096)	DFlash2, n-max 7	56.3 (0.62)	61.7 (0.70)	29.1 (0.25)	72.3 (0.85)	1.44 / 2.76
UD-IQ2_S (8,192)	n-gram (ngram-mod)	—	—	24.8 (0.38)	51.1 (0.72)	—
UD-IQ2_XXS (8,192)	none	24.6	24.8	24.6	24.6	3.27 / 3.26
UD-IQ2_XXS (8,192)	MTP sidecar, n-max 2	41.6 (0.78)	42.7 (0.82)	33.2 (0.53)	45.3 (0.90)	1.95 / 2.42
UD-IQ2_XXS (8,192)	DFlash2, n-max 7	49.8 (0.54)	53.0 (0.58)	28.7 (0.25)	67.1 (0.78)	1.63 / 2.80

Table A1: Throughput in tok/s (draft acceptance in parentheses where recorded) and GPU energy per token, every timed configuration. EXL3 rows: E7, TabbyAPI end-to-end request throughput including prefill, four prompt types, drafted rows are means over two sessions, acceptance from TabbyAPI’s log lines for the fixed-3 run. GGUF rows: E4a (IQ2_XXS), E4b (IQ2_S), E11 (n-gram), llama.cpp v0.4.0, server-timed, two repeats per prompt. Temperature 1.0, effort medium, 1,024-token cap, loaded machine; the GPU sat at 79.2–79.8 W in every row (E4a).

Prompts	Positions	Observed acc.	$p(x^*)$	$q(x^*)$	$\sum \min(p, q)$	Predicted gain	Acc. by position 0/1/2/3
prose	2,708	0.541	0.539	0.688	0.557	+0.018	0.60 / 0.47 / 0.41 / 0.23
code	2,797	0.705	0.699	0.816	0.685	-0.015	0.85 / 0.70 / 0.50 / 0.37
math	2,544	0.732	0.726	0.831	0.715	-0.011	0.86 / 0.75 / 0.60 / 0.40
rewrite	1,772	0.738	0.737	0.814	0.702	-0.035	0.93 / 0.77 / 0.53 / 0.35

Table A2: E35: per-position acceptance quantities recorded by the measurement-only patch on the daily-driver config (three 1,024-token completions per prompt type, temperature 1). $p(x^*)$ is the target’s filtered probability of the argmax draft (the stock rule’s acceptance probability), $q(x^*)$ the MTP head’s probability of it, and $\sum \min(p, q)$ the acceptance a sampled-draft rejection-sampling verifier would reach.

Prompt	2	3	4	6
prose	33.4	34.1	32.7	34.8
code	43.8	46.9	46.4	46.8
math	45.9	47.5	48.2	50.0
rewrite	44.1	48.2	49.5	48.7

Table A3: E44: end-to-end tok/s by `draft_num_tokens` (4 is the default) under dynamic drafting, daily-driver config, median of three repeats per prompt type, temperature 1. VRAM at load: 7,730 / 7,922 / 8,050 / 8,306 MiB for 2 / 3 / 4 / 6.

Config (EXL3 SC_2.00, Q8 cache)	Load	Warm
12K, no draft, 4 slots	7,794	7,834
12K, MTP, Q8 draft cache, 4 slots	10,226	10,270
8K, MTP, Q8 draft cache, 4 slots	10,098	10,142
12K, MTP, FP16 draft cache, 4 slots	10,258	10,302
12K, MTP, <code>max_batch_size: 1</code>	8,050	8,468
32K, MTP, <code>max_batch_size: 1</code>	8,754	9,588*
Bare API: no draft, 1 slot	7,260	
Bare API: MTP, 1 slot, history 4	8,050	
Bare API: MTP, 2 slots, history 4	8,732	
Bare API: no draft, 4 slots	7,914	

Table A4: Resident VRAM in MiB of the EXL3 server (E27, E28); warm follows one request (*after a 30K-token prompt). Bare-API rows are torch reserved memory from instrumented loads (`memsnap.py`).

Check	Configuration	Suite (n)	Drafted vs undrafted	Speed
E15	UD-IQ2_S + MTP n-max 3 vs undrafted, same day, same build	HumanEval+ (20); math25 lenient (25)	17/20 vs 17/20 (one discordant item each way, $p = 1.0$); 21 vs 21 (one each way)	$1.9\times$ accuracy per minute
E17	EXL3 SC_2.00 + MTP dynamic (1.4.7, 16K) vs the 2026-08-26 undrafted rows	MMLU-200	179 vs 182 (8 undrafted-only / 5 drafted-only, $p = 0.58$)	$1.6\times$ accuracy per minute
E17	UD-IQ2_S + MTP n-max 3 vs the 2026-08-26 undrafted rows	MMLU-200	177 vs 175 (7 / 9, $p = 0.80$)	$1.9\times$ accuracy per minute
E21	UD-IQ2_S + MTP n-max 3 vs the 2026-08-25 undrafted 50-item row	GSM8K, 50 paired items	46 vs 47 ($p = 1.00$)	8.3 vs 17.7s per item ($2.14\times$)

Table A5: Temperature-1 drafted-versus-undrafted quality checks that predate the hardening block (one run per item, effort medium; exact McNemar on the discordant items). At these sample sizes a two-to-three-item difference is not reliably characterized by one baseline repeat (E26); hence Section 3.3 re-tested the claim at temperature 0 on 1,000 paired items.

Comparison, A vs B (greedy, paired)	n	Acc. A / B	Discordant A / B	McNemar p	Byte-identical	Tokens A / B
E47 GSM8K-500, undrafted vs drafted	500	97.2 / 97.6%	0 / 2	0.50	318/500 = 63.6%	327 / 332
E47 MMLU-500, undrafted vs drafted	500	86.2 / 86.8%	6 / 9	0.61	105/500 = 21.0%	725 / 705
E48 GSM8K-200, UD-Q4_K_XL vs EXL3 SC_2.00	200	98.0 / 99.0%	1 / 3	0.63	—	321.9 / 322.0
E49 (i) GSM8K-100, drafted vs drafted re-run	100	99.0 / 99.0%	0 / 0	—	91/100	—
E49 GSM8K-100, undrafted vs drafted (E47 pairs)	100	98.0 / 99.0%	0 / 1	—	63/100	—
E49 (iii) GSM8K-100, undrafted vs undrafted re-run	100	98.0 / 98.0%	0 / 0	—	50/100	—
E50 GSM8K-100, undrafted vs undrafted, both clean	100	98.0 / 99.0%	0 / 1	—	64/100	—
E50 GSM8K-100, drafted mixed (E47 run) vs drafted clean	100	99.0 / 99.0%	0 / 0	—	91/100	—
E51 GSM8K-100, undrafted vs undrafted, dev 08849e3	100	97.0 / 97.0%	0 / 0	—	100/100	313.4 / 313.4
E52 GSM8K-100, undrafted vs undrafted, release 1.5.0	100	98.0 / 98.0%	0 / 0	—	100/100	323.2 / 323.2

Table A6: The hardening block (H38) in summary: temperature 0, one run per item, outputs compared by the SHA-256 of the reasoning-plus-answer text; “discordant” counts the items correct on one side only. The daily-driver config (EXL3 SC_2.00 + MTP, draft length 6 dynamic, 32K, `max_batch_size: 1`) with and without its `draft_model` block; the 4-bit reference is llama.cpp with 30 of 64 layers on the GPU. E47–E49 ran under the campaign’s mixed CUDA library set, and the E50 “clean”, E51 (exllamav3 development branch) and E52 (released exllamav3 1.5.0) runs under the fully bundled set (Section 2). Wilson intervals, hazards, wall times, and the trace probe are in Tables A7–A9.

E47: daily driver with / without its MTP drafter, greedy	GSM8K-500	MMLU-500	Pooled (1,000)
Accuracy, undrafted [Wilson 95%]	97.2% [95.4, 98.3]	86.2% [82.9, 88.9]	91.7%
Accuracy, drafted [Wilson 95%]	97.6% [95.9, 98.6]	86.8% [83.6, 89.5]	92.2%
Discordant items, undrafted-only / drafted-only	0 / 2	6 / 9	6 / 11
Exact McNemar p	0.50	0.61	0.33
Byte-identical outputs	318/500 = 63.6% [59.3, 67.7]	105/500 = 21.0% [17.7, 24.8]	423/1,000 = 42.3%
Token-count identical	68.4%	24.8%	—
Mean completion tokens, undrafted / drafted	327 / 332	725 / 705	—
Wall per item, undrafted / drafted	11.61 / 6.49 s	23.78 / 16.02 s	—
Drafted speed-up	1.79×	1.48×	—
Identity by undrafted-length tercile, short / mid / long	0.76 / 0.69 / 0.46	0.35 / 0.23 / 0.05	—
Per-token divergence hazard ε	1.6×10^{-3}	2.6×10^{-3}	—
Differing pairs: same final answer / same correctness	179 / 180 of 182	374 of 387 parsed / 380 of 395	—
Differing pairs: median length difference (drafted – undrafted)	1 token	–1 token (mean –25)	—

Table A7: E47 (H38a): the daily-driver config (EXL3 SC_2.00 + MTP, draft length 6 dynamic, 32K, `max_batch_size`: 1) against the same config with the `draft_model` block removed, temperature 0, one run per item on the $n = 500$ supersets of the campaign’s GSM8K and MMLU sets; outputs compared by the SHA-256 of the reasoning-plus-answer text. Tercile edges 227 / 339 tokens (GSM8K) and 450 / 708 (MMLU); $\varepsilon = -\ln(\text{identity})/L$ at the suite’s median undrafted length (280 and about 600 tokens). Truncations at the 4,096-token cap: MMLU 5 undrafted, 3 drafted; GSM8K none. 8.0 h of GPU.

E48, $n = 200$, greedy	UD-Q4_K_XL offloaded + MTP	EXL3 SC_2.00 + MTP	Note
Accuracy [Wilson 95%]	98.0% [95.0, 99.2]	99.0% [96.4, 99.7]	discordant 1 (4-bit only) / 3 (2-bit only); exact McNemar $p = 0.63$
Same final answer	195 of 200		the five disagreements include the four discordant items
Mean completion tokens	321.9	322.0	ratio 1.00; the 2-bit trace is > 20% longer on 34 items and > 20% shorter on 16
Wall per item	30.24 s	6.30 s	4.8× for the resident 2-bit quant; recorded, not claimed (the offload split is arbitrary)
Draft acceptance	0.84–0.85 (mean draft length 3.5)	—	llama.cpp timing lines
Truncated at 4,096 tokens	0	0	

Table A8: E48 (H38b): the 4-bit GGUF (17.6 GB; llama.cpp v0.4.0-mtpshared, 30 of 64 layers on the GPU, the shared MTP sidecar, Q8 KV, 8K context, 20 threads, 9,270 MiB resident) against the drafted 2.00-bpw daily config (the E47 drafted run, about 9.1 GiB resident), both at temperature 0, paired on the first 200 GSM8K-500 items in dataset order. 1.7 h of GPU, 6,049 s of it the offloaded 4-bit run against 1,259 s for the 2-bit side.

E49, first 100 GSM8K-500 items, greedy, mixed library set	Byte-identical [Wilson 95%]	Hazard ε per token	Accuracy (discordant)
(i) drafted vs drafted re-run	91/100 [83.8, 95.2]	3.4×10^{-4}	99.0 / 99.0 (0)
drafted vs undrafted (the E47 pairs on the same items)	63/100 [53.2, 71.8]	1.7×10^{-3}	99.0 / 98.0 (1)
(iii) undrafted vs undrafted re-run	50/100 [40.4, 59.6]	2.5×10^{-3}	98.0 / 98.0 (0)
(ii) 25 paired greedy traces, drafted vs undrafted, with top-2 log-probabilities per token (<code>divergence_probe.py</code>)			
Identical / divergent pairs	20 / 5; 23 of 25 pairs end with the same answer		
Undrafted top-1/top-2 gap at the first divergent token	0.000, 0.016, 0.016, 0.016, 0.031 nat (log-probabilities are reported in 1/64-nat steps); the drafted token is the runner-up in 5 of 5		
First divergence position	tokens 39–357, median 92 (4–61% into the trace)		
Near-tie positions in the 8,475 undrafted tokens	gap < 0.05 nat at 0.35% of positions, < 0.02 nat at 0.13%		
E50 (H38d), the same 100 items, fully bundled library set	Byte-identical [Wilson 95%]	Mean wall per item	Accuracy (discordant)
undrafted clean run 1 vs clean run 2	64/100 [54.2, 72.7]	10.81 / 10.53 s	98.0 / 99.0 (1)
undrafted clean run 1 vs mixed run A (E47) / mixed run B (E49)	58/100 / 61/100	11.74 / 9.86 s (mixed)	98.0 / 98.0 (0), both
drafted mixed (the E47 run) vs drafted clean	91/100 [83.8, 95.2]	6.39 / 5.92 s	99.0 / 99.0 (0)
drafted mixed (the E47 run) vs drafted mixed re-run, E49 (i)	91/100 [83.8, 95.2]	6.39 / 5.87 s	99.0 / 99.0 (0)

Table A9: E49 (H38c) and E50 (H38d): same-config re-runs and paired traces on the first 100 GSM8K-500 items, daily-driver config with and without the drafter, temperature 0. $\varepsilon = -\ln(\text{identity})/280$, the suite’s median undrafted length. Controls (i) and (ii) were pre-registered before either ran, (iii) after (i) had been read and before it ran, and H38d with its speed addendum on 2026-09-10 before either ran. “Mixed” is the campaign’s library set (the system cuBLASLt and NVRTC 12.1 beside torch’s bundled 12.8 libraries), “clean” the fully bundled set with LD_LIBRARY_PATH unset, the loaded libraries recorded per run (EFF-h38d*_cuda_libs.txt, no system library). The run-to-run spread of one configuration at $n = 100$ is about 8% (6.39 against 5.87s drafted), which is why the speed addendum’s $\pm 5\%$ band is not a usable test. 52 min of GPU for E49; E50 is the labels EFF-h38d_nodraft_clean1/2 and EFF-h38d_mtp_clean1.

	UD-Q4_K_XL	EXL3 SC_2.00	Ratio
math25 completion tokens (mean)	1,242	1,592	1.28 (median per item 1.16)
math25 reasoning tokens (estimate)	840	1,033	1.23
“Wait”-class transitions per math trace	2.0	4.7	2.4
explicit self-corrections (math, total)	1	4	—
tokens chosen with $p < 0.5$ (math)	2.3%	1.4%	0.6
transition-token log-probability, median	−0.22	−0.17	—
HumanEval+ completion tokens (mean)	663	647	0.98
HumanEval+ transitions per trace	1.95	1.50	0.77

Table A10: E40: temperature-0 traces with per-token log-probabilities on the same 25 math25 and 20 HumanEval+ prompts. UD-Q4_K_XL ran on llama.cpp with 30 layers on the GPU (4.8 tok/s, the 2.7-hour reference); EXL3 SC_2.00 ran drafted on TabbyAPI. Reasoning is split from the answer at `</think>`.

Ingestion interval	Cold	Repeat	Edit	@276	@1,321	@2,024	@2,639	@3,769	@4,373	Host RSS
32,768 (TabbyAPI default)	10.0	1.2	10.2	10.2	10.1	10.1	10.1	10.1	3.0	4,883 MiB
2,048	10.2	1.2	10.4	10.4	6.8	6.8	6.9	6.9	3.1	5,030 MiB
1,024 [†]	10.3	1.2	10.5	10.5	8.6	6.8	6.9	5.0	3.1	5,330 MiB

Table A11: E43: time to first token in seconds (median of two repeats) after one word is changed at the given token position of a 5,266-token document served once before, versus the recurrent checkpoint interval set by the lab patch `exl3_ckpt_interval.patch`. Daily-driver config, 36K context; VRAM 8,756 MiB in all rows. [†]512 requested, rounded up to the 1,024-token chunk.

Lever (experiment)	Config / comparator	Tokens	Accuracy, paired (discordant, p)	n , temperature
Marker penalty -2.0 (E13)	IQ2_S + MTP n3 vs none	math25 median ratio 0.94, mean -5% ; truncations 3 to 1; HE+ wall -19%	math25 lenient 21 to 22; HE+ 17 to 17	25, 20; $T=1$
Marker penalty -2.0 across tiers (E19)	EXL3 / IQ2_S / IQ2_XXS	math25 median ratio 1.03 / 0.94 / 0.85; HE+ 0.91 / 0.90 / 1.00	EXL3 GSM8K-100 98 to 94 (4/0, 0.12); IQ2_XXS math25 lenient 18 to 16	100, 25, 20; $T=1$
Penalty strength on EXL3 (E45)	-0.5 / -1.0 / -2.0 / -3.0 vs same-day baseline	math25 -7.5 / -4.6 / -5.0 / -4.8% ; GSM8K -5.2 / -5.4% at -1.0 / -2.0 ; wall -11 / -9 / -3 / -8%	math25 19 / 18 / 17 / 19 vs 19 (all $p \geq 0.5$); GSM8K 97 / 98 vs 98	25, 100; $T=1$
On-policy recalibration (E32)	own reasoning traces vs stock, undrafted 8K	HE+ $+1.5\%$; math25 -1% ; GSM8K -3 / -6% vs two baseline runs	HE+ 18 = 18; math25 18 vs 17 (1/2, 1.00); GSM8K 96 vs 98 / 95 (2/0, 0.50; 1/2, 1.00)	20, 25, 100; $T=1$
Off-policy calibration (E42)	prose + stdlib code vs self-calibrated (E32)	HE+ -1.5% ; math25 $+6.3\%$ (truncations 1 to 5); GSM8K $+15.6\%$	HE+ 17 vs 18 (1/0); math25 15 vs 18 (3/0, 0.25); GSM8K 92 vs 96 (5/1, 0.22); pooled 9:1, $p = 0.021$	20, 25, 100; $T=1$
Bit reallocation, variant A (E41)	six 1-bit MLP tensors to 2 bits at constant 2.00 bpw vs E32 conversion	HE+ -7% ; math25 $+2\%$; GSM8K $+3\%$	HE+ 17 vs 18 (2/1, 1.00); math25 15 vs 18 (3/0, 0.25); GSM8K 98 vs 96 (0/2, 0.50)	20, 25, 100; $T=1$
Sampled-draft verifier (E36)	lab patch vs stock rule	— (three $T=0$ probes identical; histograms consistent)	speed: code -5% , math -8% , rewrite -3% , prose 34.3 to 33.9 tok/s	19,074 positions; 3 repeats
Trial-answer early exit, v1 / v2 (E24, E26)	DEER proxy on the daily driver	GSM8K -24% / -18% (wall -5 / -10%); math25 -20% / -6%	GSM8K 95 / 95 vs 98 and 95 (two baseline samples); math25 18 / 18 vs 17	100, 25; $T=1$
Budget-forced exit at 90% (E31)	DEER v2 + force	math25 1,501 to 1,548	math25 18 to 15; 0 of 4 forced answers correct	25; $T=1$
reasoning_effort: low (E14)	IQ2_S + MTP n3	math25 median 1,447 to 1,263; wall -14% ; HE+ slower (23.8 vs 19.9s)	math25 lenient 21 to 22; HE+ 17 to 12	25, 20; $T=1$
SC_2.20 bpw branch (E23c)	vs SC_2.00 (+MTP), undrafted 8K	-4 to -5% on three suites; GSM8K $+4\%$	HE+ 18 = 18; math25 17 = 17; GSM8K 96 vs 98 (2/0, 0.50); MMLU-100 89 = 89	245 paired; $T=1$

Table A12: Levers against the 2-bit reasoning-length tax, each with its measured token and accuracy effect. “Discordant” is (comparator-only / lever-only) correct with the exact McNemar p . Single runs per item at the stated n ; one baseline repeat changed GSM8K-100 by three correct answers (E26), not an estimated general noise floor.

Shared config	Agents	Later-turn TTFT	Per-agent decode	Aggregate tok/s	VRAM
3 slots, 16K	1	2.4s	(rate discarded)	—	10.73 GiB
3 slots, 16K	3	27.6s*	—	—	10.91 GiB
2 slots, 24K	1	2.4s	51.3	41.4	10.33 GiB
2 slots, 24K	2	3.1s	34.3	54.1 ($\times 1.31$)	10.43 GiB
2 slots, 24K	3 (one queues)	13.7s	28.1	51.6	10.50 GiB

Table A13: E39: several synthetic agents on one TabbyAPI (EXL3 SC_2.00 + MTP dynamic, drafted slots, 16 GB recurrent stash). Each agent: a 3,725–3,758-token system prompt, six append-only turns, 700-token answers; aggregate includes prefills and gaps; temperature 1. *37–66s by turn six.

Hyp.	Pre-registered prediction	Measured outcome	<i>n</i> / evidence
H1	EXL3 MTP $\geq 1.8\times$ code/math, $\geq 1.4\times$ prose, ≤ 0.3 GiB extra VRAM	1.67–1.74 $\times$ code/math; 1.2 $\times$ prose; +1.7 GiB at 16K vs undrafted 36K and +2.4 GiB context-matched (the > 1 GiB falsifier fired)	4 prompts (E7); load figures (E27)
H2	DFlash2 $\geq 1.4\times$ prose and $\geq 15\%$ faster than MTP on prose	1.17 $\times$ prose; MTP 15–18% faster on prose (code/math $\geq 1.8\times$ held)	2 repeats $\times$ 4 prompts (E4a)
H2c	IQ2_S + DFlash2 fits at 8K	fits only at 6K (10,790 MiB); drafter costs 2,462 MiB resident	fit checks (E3, E12)
H3	-ub 512 to 128 saves ≥ 0.4 GiB	≤ 80 MiB at 8K (76 undrafted, 44 with MTP); the fit itself held on both builds	E3, E12
H5	Checkpoint tuning needed for incremental multi-turn prefill	defaults already reuse the prefix: 11 restores in 12 turns, prefill flat at 1.8–2.0 s	12 turns $\times$ 3 configs (E6)
H6	Engine versions move numbers $\geq 5\%$	llama.cpp $< 1\%$ (pp512, tg128); exllamav3 1.4.3 vs 1.4.7 -0.2 to $+1.5\%$	3 repeats (E1/E2); 4 prompts (E7)
H7	MTP n-max 2 ≥ 3 by $\geq 5\%$ on prose; p-min 0.6 $+$ $\geq 5\%$ prose	n-max 3 ≥ 2 by 3–11% on structured content, prose within noise ($+0.8$, -1.0%); p-min $+2.2\%$ on IQ2_XXS (falsified there), $+6.3\%$ on IQ2_S (met)	2 repeats (E4a/E4b)
H8	Temperature 0.7 raises acceptance $\geq 10\%$ and speed $\geq 8\%$	≤ 5 points acceptance; 1–7% speed; nothing on prose	2 repeats $\times$ 3 prompts $\times$ 2 drafters (E9)
H10	Effort low matches medium on HE+ within 1 item at $\geq 30\%$ fewer tokens	HE+ 12/20 vs 17/20 and slower (two runaway traces); math25 22 vs 21	20, 25 (E14)
H12	Undrafted EXL3 ≤ 8.0 GiB at 36,864 context	8,594–8,690 MiB at load, 9,020–9,052 after generation (below the 10.2 GiB the lab had assumed)	load figures (E7)
H15	Marker penalty $-\geq 12\%$ math25 tokens on IQ2_S	-6% paired median (-5% mean); no observed accuracy change	25 (E13)
H17	Trial-answer exit: GSM8K within 1 item; probe overhead $\leq 10\%$ of wall	-3 items (falsifier reached) but a same-config baseline re-roll also moved -3 ; overhead about 15% (v1), 9–10% (v2); math25 clause held	100, 25 (E24, E26)
H20	Batching $\times 1.7$ – 1.9 at 2 clients, $\times 2.5$ – 3.0 at 4, per-stream loss $< 25\%$	$\times 1.59$ at 2 (-19% per stream); $\times 2.12$ at 4 (-46% , past the 40% falsifier); undrafted $\times 2.61$ at 4 (-34%)	5–6 min per cell (E29)
H22	Forced answer at 90% budget: ≥ 1 correct, nothing else changes	0 of 4 forced answers correct; two pre-empted correct late finishes	25, 100 (E31)
H23	On-policy recalibration -10 – 20% tokens at equal accuracy	tokens within $\pm 6\%$; accuracy difference unresolved	20, 25, 100 paired (E32)
H25	Constant-2.00-bpw reallocation -5 – 10% tokens	tokens within $\pm 3\%$; net -2 items on 145 paired; output divergence 0.04876 vs 0.04849	145 paired (E41)

Table A14: Negative results (part 1): pre-registered predictions that were falsified or missed. Single runs per item at temperature 1 unless marked $T=0$. Outcomes describe the tested implementation and sample, not general impossibility claims.

Hyp.	Pre-registered prediction	Measured outcome	n / evidence
H26	Divergence beyond 100 tokens; lower marker log-prob on the 2-bit model; corrections not up proportionally	median divergence at token 2 (24/25 items); marker log-prob higher (-0.17 vs -0.22); corrections 1 to 4; transition clause held ($2.4\times$)	25 math, 20 code at $T=0$ (E40)
H27	3 agents at ≥ 30 tok/s, later-turn TTFT ≤ 1.5 s, $\geq 1.8\times$ aggregate, ≤ 10.5 GiB	TTFT 27.6 s; 10.91 GiB; 2 slots at 24K: 34.3 tok/s each, $\times 1.31$	6 turns per agent (E39)
H28	$\leq 15\%$ decode loss from 2K to 30K on EXL3 at 36K	-22% end-to-end (48.9 to 38.2), -21.3% decode timer (50.1 to 39.5), past the $> 20\%$ falsifier; needle 6/6 on both lanes held	513-token answers per depth (E38); 6 needle items
H29	Edit cost steps at the 2,048-token grid	steps only at ingestion checkpoints (last $2\times$ chunk of the prompt, last page); edits before 4,096 all cost the full 9.9 s	6 positions $\times$ 2 repeats (E38)
H32	Sampled-draft verifier gains ≥ 0.10 acceptance on prose	$+0.018$ prose, -0.011 to -0.035 elsewhere; $p(x^*)$ predicts observed acceptance within 0.007	9,821 positions (E35)
H33	Sampled verifier lossless and $+10$ – 25% prose tok/s	small correctness checks passed; prose 34.3 to 33.9 tok/s; code -5% , math -8% , rewrite -3%	19,074 positions; 3 repeats (E36)
H34	fp16 GEMM at the model’s shapes ≤ 32 TFLOPS at the cap	33.5–61.8 TFLOPS on six of seven shapes (26.5 on attn_kv); GEMM share 74% as predicted	1 profiled prefill (E37)
H36	Penalty token reduction monotone in strength (-8 / -20 / -30%)	flat about -5% from -0.5 to -3.0 , largest at -0.5 (-7.5%); accuracy clause held	25, 100 (E45)
H37	Draft-length optimum at 3–4 (code/math), 2–3 (prose); 6 worst	6 fastest on prose/math, 3 on code, 4 on rewrite; 2 slowest except prose; spread 6–11%	3 repeats $\times$ 4 prompts (E44)
H38a	Drafted vs undrafted byte-identical on $\geq 95\%$ of 1,000 greedy items (identity clause)	63.6% and 21.0% byte-identical (GSM8K, MMLU), past the $< 90\%$ falsifier; hazard 1.6 – 2.6×10^{-3} /token; accuracy clause held (97.6 vs 97.2, 86.8 vs 86.2; $p \geq 0.50$)	1,000 paired at $T=0$ (E47)
H38c (iii)	Undrafted same-config re-run $\geq 97\%$ byte-identical, 100 GSM8K items	50/100 [40.4, 59.6], past the $\leq 90\%$ falsifier; all 50 differing pairs reach the same answer; drafted re-run (i) 91/100, between prediction and falsifier	100 at $T=0$ (E49)
H38d	Undrafted same-config re-run $\geq 97\%$ byte-identical under a fully bundled CUDA library set; drafted wall per item within $\pm 5\%$ of the mixed-set 6.49 s	64/100 [54.2, 72.7]: prediction not met, the $\leq 60\%$ falsifier missed by four items; drafted clean 5.92 s vs 6.39 / 5.87 s mixed (inside run-to-run variance, so the $\pm 5\%$ band is not a usable test); drafted clean vs mixed 91/100	100 at $T=0$ (E50)

Table A14: Negative results (continued): pre-registered predictions that were falsified or missed, with the measured outcome and its sample; predictions from the pre-registration file, outcomes from the campaign document’s per-clause verdicts. Single runs per item at temperature 1 unless marked $T=0$.